Supplementary Material: Computer-Assisted Accuracy Proof

Contents

1 Computer-assisted accuracy proof **2**

1.1 Statement and scope of the certificate . . . 2
1.2 Overview of the proof . . . 2
1.3 The trust boundary between the computer assisted calculation in Arb-lib and Lean . 3
1.4 yH . . . 3
1.5 Interval Taylor data . . . 3
1.6 Path A: exact finite Fourier algebra . . . 3
1.7 Path B: certified discrete Hilbert transform . . . 4
1.8 Interval Taylor data . . . 5
1.9 Positivity of the capillary denominator . . . 6
1.10 Higher derivatives of D and G . . . 6
1.11 Residual assembly and passage from the grid to the torus . . . 7
1.12 Julia/Arb implementation . . . 7
1.13 Memory-sharded computation . . . 8
1.14 Lean definitions and reusable analytic library . . . 8
1.15 Exact certificate data and convention alignment . . . 9
1.16 The numerical trust boundary in Lean . . . 10
1.17 Numerical result and interpretation of precision . . . 10
1.18 Reproducibility and file map . . . 11

1 Computer-assisted accuracy proof

1.1 Statement and scope of the certificate

The purpose of the computer-assisted calculation is to certify the accuracy of an explicit numerical traveling-wave profile. After the search and polishing stages, the profile is frozen as the finite cosine polynomial

$$y_N(u) = \frac{a_0}{2} + \sum_{k=1}^{N} a_k \cos(ku), \qquad N = 150. \tag{1}$$

The coefficients in Eq. (1) are fixed binary floating-point numbers. The verification treats their exact dyadic values as the definition of the candidate. The neural network is therefore a search tool, not part of the object appearing in the theorem.

Let H denote the 2π-periodic Hilbert transform with convention

$$H[\cos(ku)] = \sin(ku), \qquad H[\sin(ku)] = -\cos(ku), \qquad H[1] = 0. \tag{2}$$

For

$$D = \sqrt{y_u^2 + \big(1 + H[y_u]\big)^2}, \qquad G = \frac{1 + H[y_u]}{D}, \tag{3}$$

the residual evaluated by the verifier is

$$R(y) = -\mathcal{B}(y) = \mu H[y_u] - y - yH[y_u] - H[yy_u] + \beta\, \partial_u\Big(\frac{y_u}{D}\Big) - \beta H[\partial_u G]. \tag{4}$$

The parameters for which we verify the residual in this work are:

$$\mu = 1.3660618372350841, \text{ and } \beta = 0.248004. \tag{5}$$

The certificate later records those stored values as exact dyadic rationals.

The calculation proves two continuum inequalities:

$$\inf_{u\in\mathbb{R}} D(u) \geq d_* > 0, \qquad \sup_{u\in\mathbb{R}} |R(y_N)(u)| \leq \varepsilon_*. \tag{6}$$

Specifficaly for the profile presented in the paper $N = 150$ and

$$D(u) \geq 0.3446588, \qquad |R(y_N)(u)| \leq 1.78 \times 10^{-13} \quad \text{for every } u \in \mathbb{R}. \tag{7}$$

Consequently Eq. (7) is also the claimed bound for $|\mathcal{B}(y_{150})|$.

This is an accuracy theorem for a supplied finite series. It is not an existence or uniqueness theorem for an exact nearby water wave. Such a result would additionally require a rigorous inverse bound for the linearized Babenko operator and a Newton–Kantorovich or contraction argument; neither ingredient is part of the present certificate.

1.2 Overview of the proof

The proof has three conceptual layers. First, finite Fourier algebra constructs the candidate and handles the Hilbert transform whenever its input is a known trigonometric polynomial. Second, Julia and Arb compute outward-rounded interval enclosures for the nonlinear residual, add analytic corrections for unresolved Fourier modes, and use Taylor's theorem to pass from a grid to the whole torus. Third, Lean defines the same mathematical objects, proves the reusable analytic lemmas underlying the calculation, and checks the exact logical consequences of the exported numerical certificate.

The Hilbert-transform work is divided into two pathways.

Path A: finite trigonometric algebra. The input has a finite Fourier representation, so H acts mode-by-mode using Eq. (2). No sampling or aliasing correction is needed for this operation.

Path B: certified sampled transform. The input is represented by interval-enclosed values and derivatives on a uniform grid, rather than by a finite exact coefficient vector. A discrete Hilbert transform is evaluated by certified convolution, and a rigorous Fourier-tail estimate is added to enclose the continuous transform.

Path B is essential for $H[\partial_u G]$, because the square root and reciprocal in Eq. (3) generally give G an infinite Fourier series. The current Julia implementation also routes $H[yy_u]$ through Path B for uniformity, although yy_u is itself a finite trigonometric polynomial of degree at most 2N.

1.3 The trust boundary between the computer assisted calculation in Arb-lib and Lean

1.4 yH

1.5 Interval Taylor data

An Arb ball $[m \pm r]$ represents the closed interval $[m - r, m + r]$. Elementary operations such as addition, multiplication, division away from zero, square root on a positive interval, sine, and cosine are rounded outward. Repeated use of the interval inclusion principle then guarantees that the final ball contains the exact result for the supplied dyadic inputs.

The rigor of the full verifier is not automatic from pointwise ball arithmetic alone. The code also supplies mathematical bounds for variation between grid points, unresolved Fourier modes, and nonlinear derivative growth. These analytic corrections are computed as Arb balls and inserted explicitly into the enclosure.

We outline here the estimate procedure of four terms in the residual in increasing order of complexity: $yH[y_u]$ which is estimated through path A, and $H[vy_u]$ and the Capillarity terms that are estimated through path B.

1.6 Path A: exact finite Fourier algebra

For a trigonometric polynomial

$$F(u) = \frac{\alpha_0}{2} + \sum_{k=1}^{K} \big(\alpha_k \cos(ku) + \gamma_k \sin(ku)\big), \tag{8}$$

Path A constructs

$$H[F](u) = \sum_{k=1}^{K} \big(\alpha_k \sin(ku) - \gamma_k \cos(ku)\big). \tag{9}$$

Differentiation and multiplication are likewise carried out by finite coefficient formulas.

Applying Eq. (9) to

$$y_u(u) = -\sum_{k=1}^{N} k a_k \sin(ku) \tag{10}$$

gives

$$H[y_u](u) = \sum_{k=1}^{N} k a_k \cos(ku). \tag{11}$$

This explicit function is denoted H_y in the implementation. It is used directly in D, G, and the local product yH_y.

For any derivative order $p \geq 1$, the same coefficient data give the rigorous global estimate

$$\|F^{(p)}\|_\infty \leq \sum_{k=1}^{K} k^p \big(|\alpha_k| + |\gamma_k|\big), \tag{12}$$

with the constant term included separately when $p = 0$.

These finite sums are evaluated with outward rounding and seed the derivative-bound chain used by both pathways.

These finite sums are evaluated with outward rounding and seed the derivative-bound chain used by both pathways.

The corresponding Lean files are

`Definitions.lean`, `HilbertOnTrigPoly.lean`.

The first defines the Fourier-multiplier operator. The second proves the sine and cosine rules, compatibility with finite sums, and the bridge theorem

`periodicHilbert_yu_eq_Hyu`.

Thus the explicit finite expression in Eq. (11) is proved to equal the abstract Fourier-defined Hilbert transform of y_u.

1.7 Path B: certified discrete Hilbert transform

Let F be a smooth real 2π-periodic function and let $u_j = 2\pi j/N_{\text{grid}}$, where N_{grid} is even. The discrete Hilbert transform has Fourier multipliers $-i$ on positive discrete frequencies, $+i$ on negative frequencies, and zero at the constant and Nyquist modes. Julia realizes this operator as the circular convolution

$$H_{N_{\text{grid}}}[F](u_j) = \sum_{n=0}^{N_{\text{grid}}-1} h_n F(u_{j-n \bmod N_{\text{grid}}}), \tag{13}$$

with

$$h_n = \begin{cases} \dfrac{2}{N_{\text{grid}}} \cot\left(\dfrac{\pi n}{N_{\text{grid}}}\right), & n \text{ odd}, \\ 0, & n \text{ even}. \end{cases} \tag{14}$$

The cotangent formula is not a second definition of the continuous Hilbert transform. It is the inverse discrete Fourier transform of the discrete multiplier, and therefore an efficient real-space implementation of the same finite-dimensional operator.

The circular convolution is computed as a linear polynomial convolution followed by folding modulo $x^{N_{\text{grid}}} - 1$. Arb complex-ball polynomial multiplication encloses all arithmetic and transform rounding. The use of a fast convolution algorithm affects runtime, not the operator or the proof.

The sampled transform still differs from the continuous Hilbert transform because Fourier modes separated by a multiple of N_{grid} have the same samples. Write

$$\widehat{F}_k = \frac{1}{2\pi} \int_0^{2\pi} F(u) e^{-iku}\, du. \tag{15}$$

If F is periodic and $\|F^{(p)}\|_\infty \le C_p$ for an integer $p \ge 2$, integration by parts p times gives

$$|\widehat{F}_k| \le \frac{C_p}{|k|^p}, \qquad k \neq 0. \tag{16}$$

The discrete Fourier coefficient at a principal frequency is the sum of all continuous coefficients aliased into that class. The continuous and discrete Hilbert multipliers differ by at most two on an aliased mode. Combining this observation with Eq. (16), the two-sided Fourier tail, and the integral test yields

$$\left|H[F](u_j) - H_{N_{\mathrm{grid}}}[F](u_j)\right| \le \varepsilon_{\mathrm{alias}}(F) := \frac{4C_p}{(p-1)(N_{\mathrm{grid}}/2-1)^{p-1}}. \tag{17}$$

The symmetric interval $[-\varepsilon_{\mathrm{alias}}, \varepsilon_{\mathrm{alias}}]$ is added to every output ball.

The transform is applied at every derivative level needed by the final Taylor argument. Since the periodic Hilbert transform commutes with differentiation under the smoothness hypotheses used here,

$$\partial_u^m H[F] = H[\partial_u^m F], \tag{18}$$

the same calculation supplies local enclosures of all required derivatives. Global derivative bounds for the output use

$$\|H[F]\|_\infty \le \frac{\pi^2}{3}\|F''\|_\infty, \tag{19}$$

which follows by two integrations by parts and $2\sum_{k\ge1} k^{-2} = \pi^2/3$.

The Lean file `AliasingErrorBound.lean` proves the DFT sampling identity, Fourier-decay and tail estimates, the bound Eq. (17), and the equivalence between the multiplier form and the cotangent convolution in Eq. (13). The file `FourierTailHilbert.lean` proves Eq. (19) and the supporting Fourier/Hilbert facts. Finally, `FFTHilbertCorrectness.lean` packages these results into correctness theorems for one transform and for the derivative-indexed Taylor data returned by Julia.

1.8 Interval Taylor data

An Arb ball $[m \pm r]$ represents the closed interval $[m-r, m+r]$. Elementary operations such as addition, multiplication, division away from zero, square root on a positive interval, sine, and cosine are rounded outward. Repeated use of the interval inclusion principle then guarantees that the final ball contains the exact result for the supplied dyadic inputs.

The rigor of the full verifier is not automatic from pointwise ball arithmetic alone. The code also supplies mathematical bounds for variation between grid points, unresolved Fourier modes, and nonlinear derivative growth. These analytic corrections are computed as Arb balls and inserted explicitly into the enclosure.

For a function F, Julia's `TaylorSeries` object stores

$$F^{(m)}(u_j) \in I_{j,m}(F), \quad 0 \le m \le M, \qquad C_m(F) \ge \|F^{(m)}\|_\infty, \quad 0 \le m \le M+1. \tag{20}$$

Products use the exact Leibniz identity

$$(FQ)^{(m)} = \sum_{r=0}^{m} \binom{m}{r} F^{(r)} Q^{(m-r)}, \tag{21}$$

both for local derivative balls and for global bounds. Reciprocals and square roots are propagated through Faà di Bruno formulas. A downward grid-and-Lipschitz pass can tighten an existing bound by

$$\|F^{(m)}\|_\infty \le \max_j \sup\{|x| : x \in I_{j,m}(F)\} + hC_{m+1}(F), \qquad h = \frac{2\pi}{N_{\mathrm{grid}}}. \tag{22}$$

The new bound is retained only when its outward upper endpoint improves the previous rigorous estimate.

1.9 Positivity of the capillary denominator

The reciprocal terms are not evaluated until D has been proved positive. Set

$$A = y_u, \qquad B = 1 + H[y_u], \qquad D = \sqrt{A^2 + B^2}. \tag{23}$$

At each grid point, Arb computes a lower endpoint $\underline{D}_j$ for $D(u_j)$. The reverse triangle inequality gives

$$|D(u) - D(v)| \le |A(u) - A(v)| + |B(u) - B(v)|. \tag{24}$$

Consequently, if $L_A \ge \|A'\|_\infty$ and $L_B \ge \|B'\|_\infty$, then

$$d_{\mathrm{Lip}} := \min_j \big(\underline{D}_j - h(L_A + L_B)\big) \le \inf_u D(u). \tag{25}$$

This proof neither differentiates nor divides by D, so it does not presuppose the positivity it establishes. The computation stops unless outward-rounded arithmetic proves $d_{\mathrm{Lip}} > 0$.

After positivity is known, Julia constructs Taylor data for D and obtains a second lower bound from the local derivatives and the Taylor remainder. The implementation selects the ball having the larger rigorous lower endpoint. Both candidates are valid lower bounds; selecting the larger one only sharpens the subsequent reciprocal estimates.

1.10 Higher derivatives of D and G

Direct Faà di Bruno estimates for many derivatives of a square root and reciprocal can become unnecessarily large. The verifier therefore also uses algebraic recurrences. Let $S = A^2 + B^2 = D^2$. Differentiating $D^2 = S$ m times and isolating the two endpoint terms gives

$$2DD^{(m)} = S^{(m)} - \sum_{r=1}^{m-1} \binom{m}{r} D^{(r)} D^{(m-r)}. \tag{26}$$

If $D \ge d > 0$, then

$$\|D^{(m)}\|_\infty \le \frac{1}{2d} \left(\|S^{(m)}\|_\infty + \sum_{r=1}^{m-1} \binom{m}{r} \|D^{(r)}\|_\infty \|D^{(m-r)}\|_\infty \right). \tag{27}$$

Similarly, the identity $GD = B$ gives

$$G^{(m)} D = B^{(m)} - \sum_{r=0}^{m-1} \binom{m}{r} G^{(r)} D^{(m-r)}, \tag{28}$$

and therefore

$$\|G^{(m)}\|_\infty \le \frac{1}{d} \left(\|B^{(m)}\|_\infty + \sum_{r=0}^{m-1} \binom{m}{r} \|G^{(r)}\|_\infty \|D^{(m-r)}\|_\infty \right). \tag{29}$$

The quantities S and B are finite trigonometric polynomials, so their derivative bounds come directly from Eq. (12). The recurrence avoids replacing D or G by a finite Fourier series. Julia compares these algebraic bounds with the valid composition-based bounds and uses the tighter outward endpoint where needed for the top derivatives in Path B.

1.11 Residual assembly and passage from the grid to the torus

The local interval records are composed in the same order as Eq. (4): finite data for y, y_u, and $H[y_u]$; Leibniz products; the Path-B transform of yy_u; construction of D, D^{-1}, and G; differentiation of y_u/D and G; the Path-B transform of $\partial_u G$; and the final linear combination. For every grid node and every $0 \le m \le M$, the result is an interval

$$R^{(m)}(u_j) \in I_{j,m}(R), \tag{30}$$

together with $C_{M+1}(R) \ge \|R^{(M+1)}\|_\infty$.

For $u = u_j + t$, $0 \le t \le h$, Taylor's theorem yields

$$|R(u)| \le \sum_{m=0}^{M} \frac{U_{j,m}(R)h^m}{m!} + \frac{C_{M+1}(R)h^{M+1}}{(M+1)!}, \tag{31}$$

where $U_{j,m}(R) = \sup\{|x| : x \in I_{j,m}(R)\}$. The outward-rounded maximum of the right-hand side over all cells is the certified ε_*. This is the step that converts finitely many computations into a statement about every point of the continuous periodic domain. With $M = 6$, the final remainder scales as h^7.

1.12 Julia/Arb implementation

The package `BabenkoRigorous` implements the numerical proof. Its principal files have the following roles.

`FourierSeries.jl`. Defines the finite `TrigPoly` representation and the Path-A operations: evaluation, differentiation, Hilbert transform, multiplication, and coefficient-based supremum bounds.

`TaylorSeries.jl`. Defines local derivative matrices and global derivative-bound vectors; implements Taylor supremum and infimum scans, Leibniz products, reciprocal and square-root composition, downward bound tightening, denominator positivity, and the algebraic D/G recurrences.

`GDerivatives.jl`. Constructs the Taylor representation of $G = (1 + H[y_u])/D$ from the certified denominator and reciprocal.

`FFTHilbert.jl`. Implements Path B by Arb complex-polynomial convolution with the cotangent kernel, adds Eq. (17) to each output ball, and lifts the transform through all derivative levels using Eq. (18) and the global bound Eq. (19).

`ResidualBound.jl`. Assembles the complete gravity–capillary residual and applies the final Taylor scan. In the implemented route, $H[yy_u]$ and $H[\partial_u G]$ both pass through `fft_hilbert_series`; only the latter intrinsically requires the non-polynomial pathway.

`ShardStore.jl` **and** `Sharded.jl`. Reorganize the same computation into exact on-disk shards so that a large grid does not require all Taylor matrices to be resident simultaneously.

`Certificate.jl`. Converts binary inputs and outward endpoints to exact rational literals and emits the generated Lean data module. The current formatter emits no axioms; the two numerical anchors live in a separate certificate-specific Lean module.

Most local arithmetic operations are ordinary point operations lifted to Arb balls, but the proof also depends on the explicitly programmed analytic corrections in Eqs. (17), (22), (25), and (31). Arb automatically encloses numerical rounding; it does not invent these mathematical bounds.

1.13 Memory-sharded computation

At $N_{\text{grid}} = 2{,}000{,}000$, storing every intermediate matrix at once is unnecessarily expensive. The sharded implementation exploits the structure of the calculation:

1. Compute coefficient-level derivative bounds once from the finite trigonometric data.
2. Pointwise nonlinear recurrences and min/max reductions are streamed over contiguous grid shards.
3. Each global Hilbert transform is processed one derivative column at a time. Exact Arb column slices are stored on disk.
4. The final residual is reassembled shard by shard and the Taylor-cell maximum is reduced in ascending grid order.

Intermediate balls are serialized with Arb's exact dump/load representation, including mantissa, exponent, and radius; decimal text is not used for this round trip. The reduction order matches the monolithic loops, and repository tests compare the sharded and monolithic paths on smaller grids. Sharding therefore changes memory use and scheduling, not the mathematical enclosure.

The successful two-million-point calculation used shards of 1024 points. The first stage ran on eight workers with four Julia threads per worker; the main transform and final stages used eight threads. The stages consisted of parallel pointwise reductions, merging of the worker states, derivative-by-derivative certified convolutions, and a final sharded residual scan. Separate operating-system processes between major stages released the memory associated with completed stages and left exact state dumps for reproducibility.

1.14 Lean definitions and reusable analytic library

The Lean project `WaveCAP` defines the mathematical operator rather than treating it as an uninterpreted symbol. For a real function f, `Definitions.lean` sets

$$H[f](u) = \operatorname{Re} \sum_{n \in \mathbb{Z}} \big(-i \operatorname{sgn}(n)\big) \widehat{f}_n e^{inu}, \tag{32}$$

with multiplier zero at $n = 0$. The same file defines the finite profile, its derivative, the explicit H_y, D, G, and R from Eq. (4). The coefficient type $\mathrm{Fin}(N) \to \mathbb{R}$ makes every profile sum finite.

The reusable formal library mirrors the proof architecture:

- **Hilbert transform.** `HilbertOnTrigPoly.lean` proves Path A and $H[y_u] = H_y$. `FourierTailHilbert.lean` proves the $\pi^2/3$ estimate and mode facts. `HilbertCommutesDeriv.lean` proves derivative commutation.
- **Aliasing and Path B.** `AliasingErrorBound.lean` proves the sampled DFT aliasing theorem and the cot-kernel equivalence. The file `FFTHilbertCorrectness.lean` states the correctness contract for the interval transform and its Taylor-series lift.
- **Derivative calculus.** `LeibnizBound.lean` and `FaaDiBrunoBound.lean` prove the product and composition bounds used by Julia.
- **Taylor data.** `TaylorRemainder.lean` and `GridToSupNormTaylor.lean` prove the local remainder and the grid-to-continuum theorem. The file `TaylorSeriesCorrectness.lean`

defines the semantic invariant `IsTaylorSeriesOf` and proves that the arithmetic operations preserve it. `SeriesCorrectness.lean` covers the square-root and reciprocal recurrences, while `TightenSupBoundsLemma.lean` justifies the downward tightening pass.

- **Denominator and capillary chain.** `DminBoundCorrectness.lean` formalizes the positivity calculation, and `TightDGSupBounds.lean` proves Eqs. (27)–(29).
- **Residual assembly.** `ResidualAssemblyCorrectness.lean` composes the Path-A and Path-B records into the full residual, proves periodicity, and proves the general extension of a one-period bound to every real argument.

These are generic theorems about the mathematics and data-structure contracts of the pipeline. They are useful independently of the particular numerical profile.

1.15 Exact certificate data and convention alignment

The generated module

`CertificatePhase4ScalarN150Phase420260827HalfDC.lean`

contains 151 exact rational coefficients and exact rational representations of μ, β, N_{grid}, M, the residual upper endpoint, and the denominator lower endpoint. In particular,

$$\mu_{\mathrm{cert}} = \frac{1538048895284245}{1125899906842624}, \tag{33}$$

$$\beta_{\mathrm{cert}} = \frac{2233821443972785}{9007199254740992}, \tag{34}$$

and

$$\varepsilon_{0,\mathrm{cert}} = \frac{66570650447636094343693612891855023841}{37414441915671114706014331717536845303191871 8731001856}, \tag{35}$$

$$D_{\mathrm{lower,cert}} = \frac{3002401899956962760047997646878784596058 5}{87112285931760246646623899502532662132736}. \tag{36}$$

Thus moving the certificate into Lean introduces no decimal-rounding step.

Julia and Lean use different storage conventions for the constant cosine mode. Julia evaluates the first source entry as $a_0/2$, while the generic Lean function `y` uses its zeroth stored entry directly. The certificate therefore stores

$$\widehat{a}_0 = a_0/2, \qquad \widehat{a}_k = a_k \quad (k \geq 1). \tag{37}$$

The generic module `HalfDCConvention.lean` proves the two coefficient cases and proves that this adapter preserves y_u, H_y, D, and G. The certificate-specific theorem checks the exact zeroth-entry identity. The source vector is not also encoded as a second complete Lean vector, so the present formal theorem does not compare two named 151-component vectors; the generated certificate already contains the converted vector.

The audit module `AuditPhase4ScalarN150HalfDC.lean` separately proves the sign identity in Eq. (**??**) and the corresponding equality of absolute values. Both the half-DC identities and the residual sign bridge are independent of the numerical trust anchors.

1.16 The numerical trust boundary in Lean

The present certificate does not serialize the millions of local derivative balls, all global derivative vectors, or the intermediate convolution enclosures needed for Lean to reconstruct the two-million-point run. The certificate-specific numerical interface therefore consists of exactly two assumptions recording the final Arb conclusions:

$$\mathsf{A}_D: \quad D_{\mathrm{lower,cert}} > 0 \quad \text{and} \quad \forall u \in \mathbb{R}, \quad D_{\mathrm{lower,cert}} \le D(a_{\mathrm{cert}}; u), \tag{38}$$

$$\mathsf{A}_R: \quad \mathsf{A}_D \Longrightarrow \forall u \in [0, 2\pi], \quad |R(a_{\mathrm{cert}}, \mu_{\mathrm{cert}}, \beta_{\mathrm{cert}}; u)| \le \varepsilon_{0,\mathrm{cert}}. \tag{39}$$

Their Lean names are

`cert_arb_dmin_bound`, `cert_arb_sup_bound_le_eps0`.

They are declared in

`CertificatePhase4ScalarN150Phase420260827HalfDCAxioms.lean`.

This distinction is important. Lean proves the universal analytic ingredients used by the verifier, but the current certificate-facing theorem does not instantiate those generic results from serialized grid matrices. Instead, it imports Eqs. (38)– (39) as the explicit numerical trust boundary. Accordingly, the result should be described as a machine-checked theorem with two externally computed Arb anchors, not as a replay of every interval operation inside Lean.

From the two anchors Lean performs the remaining deductions:

1. It projects the global denominator lower bound from A_D.
2. It applies A_R on $[0, 2\pi]$.
3. It proves periodicity of the residual and extends the bound to all $u \in \mathbb{R}$.
4. It verifies by exact rational normalization that $\varepsilon_{0,\mathrm{cert}} < 178/10^{15}$ and $3446588/10^7 < D_{\mathrm{lower,cert}}$.
5. It transfers the absolute residual theorem to the manuscript operator using Eq. (**??**).

The corresponding public modules are

`MainPhase4ScalarN150Phase420260827HalfDC.lean`,
`AuditPhase4ScalarN150HalfDC.lean`.

The axiom audit for the exact and decimal residual theorems reports the standard Mathlib principles `propext`, `Classical.choice`, and `Quot.sound`, together with precisely the two numerical anchors above. The denominator theorem uses only the first numerical anchor. The half-DC and sign-alignment results use no certificate-specific numerical axioms.

1.17 Numerical result and interpretation of precision

The successful Crunchy2 computation used

$$N = 150, \qquad N_{\mathrm{grid}} = 2{,}000{,}000, \tag{40}$$

$$M = 6, \qquad p_{\mathrm{Arb}} = 128 \text{ bits}, \tag{41}$$

with shard size 1024. The final Arb ball was reported as

$$\mathrm{mid}(B_R) = 1.9928311263803702 \times 10^{-14}, \tag{42}$$

$$\mathrm{rad}(B_R) = 1.5799937398052128 \times 10^{-13}, \tag{43}$$

$$\inf B_R = -1.3807106271671758 \times 10^{-13}, \tag{44}$$

$$\sup B_R = 1.7792768524432498 \times 10^{-13}. \tag{45}$$

Here B_R is the ball returned by the supremum-bound computation; it is not an enclosure of the range of R, which explains why its lower endpoint may be negative. The theorem uses the outward upper endpoint $1.7792768524432498 \times 10^{-13}$, not the midpoint or radius separately. The denominator calculation returned the outward lower endpoint

$$D_{\mathrm{lower}} = 3.4465883518530394 \times 10^{-1}. \tag{46}$$

The run log ends with `ALL_STAGES_COMPLETE`, and the final stage has zero exit status.

The phrase "near machine precision" refers here to the practical accuracy floor of the double-precision candidate and the complete nonlinear residual, not to the rounding unit of the 128-bit Arb calculation. IEEE double precision has machine epsilon $2^{-52} \approx 2.22 \times 10^{-16}$ and unit roundoff $2^{-53} \approx 1.11 \times 10^{-16}$. Cancellation, derivative amplification, conditioning, and the storage of the candidate coefficients can raise the attainable residual above these raw constants. The 128-bit Arb precision, roughly 38 decimal digits, makes arithmetic rounding in the verification negligible relative to the analytic Taylor and aliasing corrections; it does not force the final certificate radius to be 10^{-38}.

1.18 Reproducibility and file map

The archived run directory

`BabenkoRigorous/results/phase4_scalar_n150_20260827_2m_20260829`

contains the run description and checksums for the raw logs, stage logs, coefficient file, Arb endpoint dumps, scripts, and generated Lean certificate. The installed half-DC certificate was compared byte-for-byte with the generated artifact. The dedicated Lean theorem and audit modules were checked directly with the pinned Lean and Mathlib version `v4.29.0-rc6`.

The principal archived artifacts have the following SHA-256 checksums:

```
4d65357a64a0ccfbf0ed67f8dd0e1970f0ceb549a8cbf647d8490e4ac48abb07
ee8fe5db1ea3bbd6e37d0888fdadc00c9c9e1d6ce81cb9d46ae3e4db8ffcdf43
```

for the run archive and the half-DC alignment archive, respectively. The installed generated half-DC Lean certificate has SHA-256 checksum

```
2190a52f3520ab0b15683eab592ee1105797d737a22f2e24a869130f5a431ed2
```

For readers following the implementation, the Julia source names below are relative to `BabenkoRigorous/`. The Lean source names are relative to `WaveCAP/WaveCAP/`.

- **Path A.** Julia: `src/FourierSeries.jl`. Lean: `Definitions.lean` and `HilbertOnTrigPoly.lean`.

- **Path B and the cot kernel.** Julia: `src/FFTHilbert.jl`. Lean: `AliasingErrorBound.lean`, `FourierTailHilbert.lean`, and `FFTHilbertCorrectness.lean`.
- **Nonlinear Taylor data and denominator.** Julia: `src/TaylorSeries.jl` and `src/GDerivatives.jl`. Lean: `TaylorSeriesCorrectness.lean`, `SeriesCorrectness.lean`, `DminBoundCorrectness.lean`, and `TightDGSupBounds.lean`.
- **Residual and continuum bound.** Julia: `src/ResidualBound.jl`. Lean: `TaylorRemainder.lean`, `GridToSupNormTaylor.lean`, and `ResidualAssemblyCorrectness.lean`.
- **Large-grid organization.** Julia: `src/ShardStore.jl`, `src/Sharded.jl`, and `scripts/stage_run_phase4_scalar_n150_20260827.jl`.
- **Certificate and theorem.** Julia: `src/Certificate.jl`. Lean: the generated half-DC certificate, its certificate-specific axiom module, the corresponding `Main` module, and `AuditPhase4ScalarN150HalfDC.lean`.

These components yield a reproducible continuum residual bound for the profile in Eq. (1). They state the mathematical pathways, numerical enclosures, formal definitions, and remaining numerical trust boundary separately.